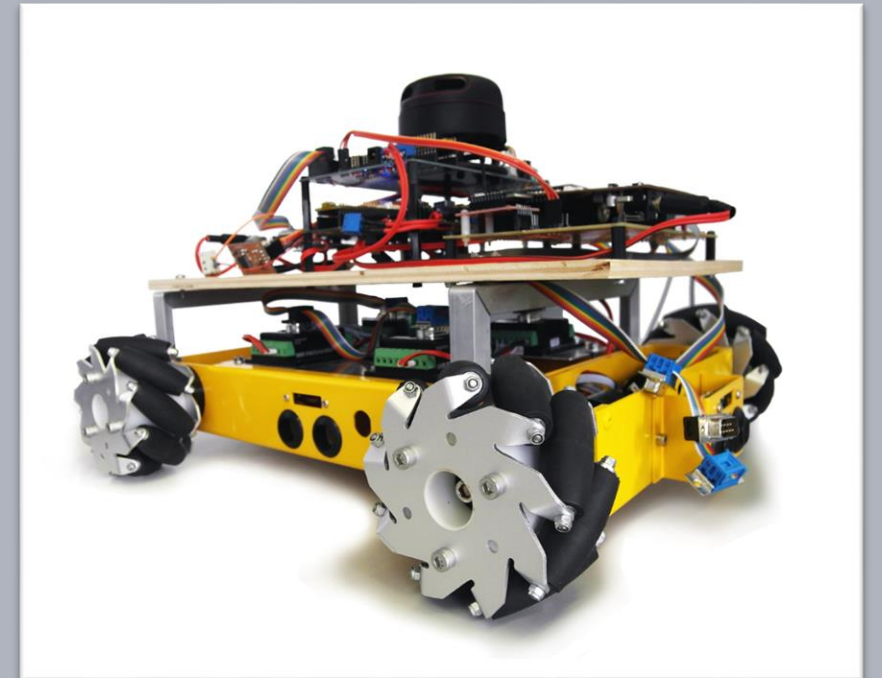


Advanced Embedded Architectures and Applications

Communication and Protocol Design

Prof. Dr.-Ing. Peter Fromm



Content

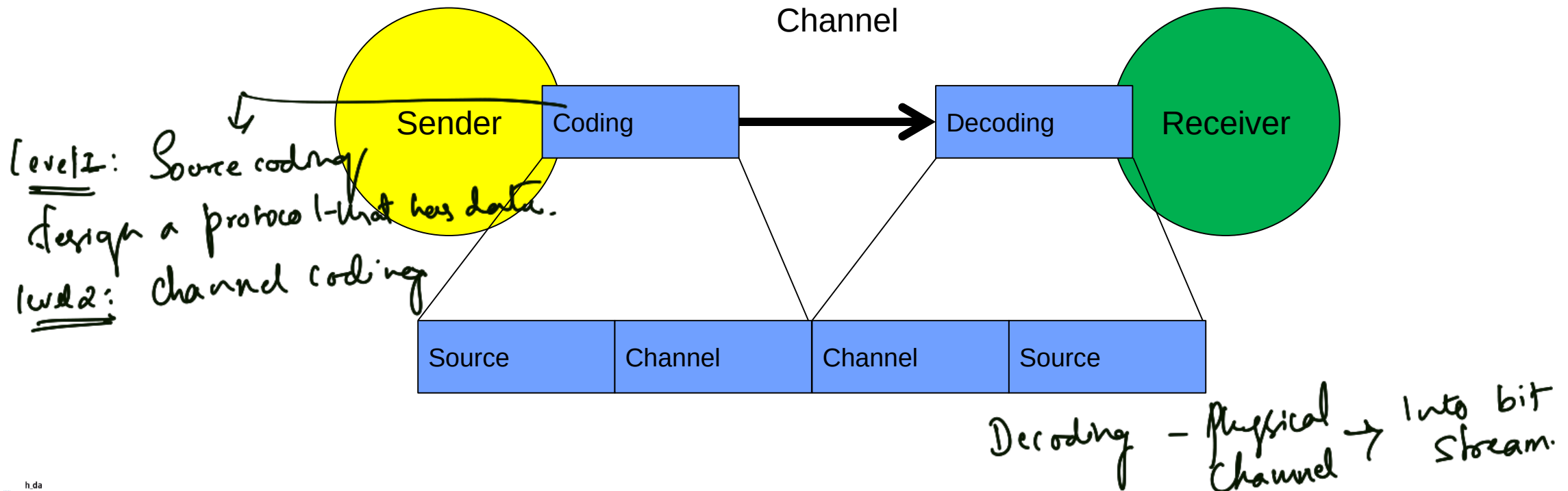
- Communication fundamentals
- Challenges from a developer perspective
- Requirements for our car
- Zigbee wireless communication
- Low level driver
- Protocol Stack
- Interfacing the signallayer



Telecommunication

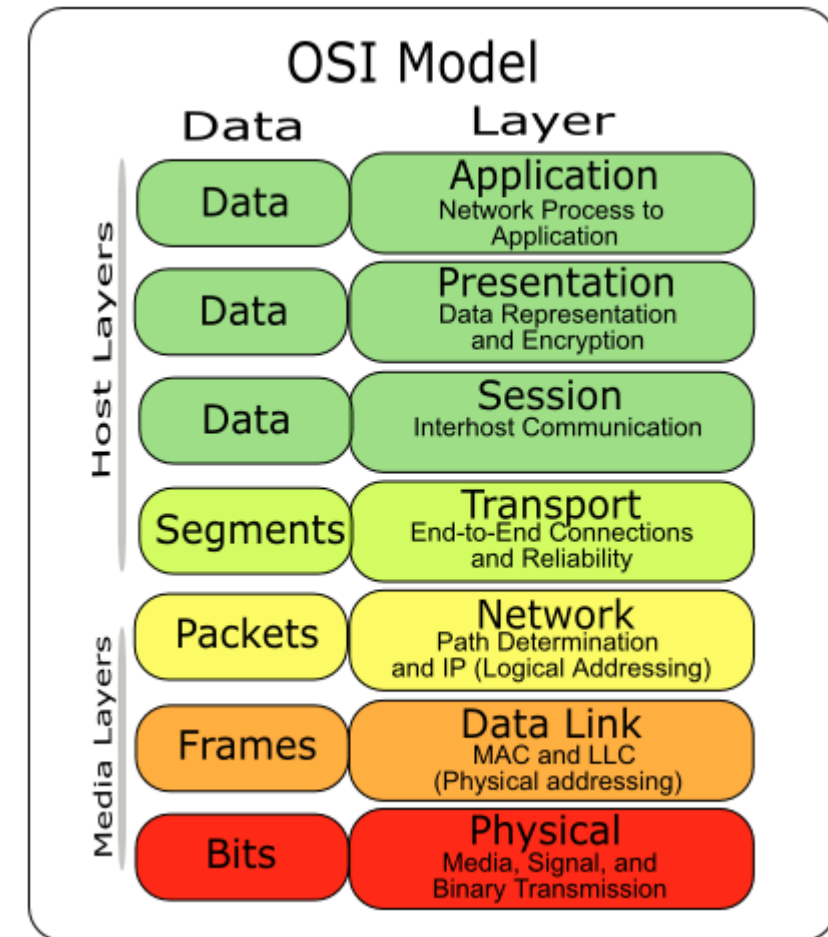
Telecommunication is the exchange of signs, signals, messages, words, writings, images and sounds or information of any nature by wire, radio, optical or other electromagnetic systems.... It is transmitted through a transmission medium, such as over physical media, for example, over electrical cable, or via electromagnetic radiation through space such as radio or light...

(Wikipedia)

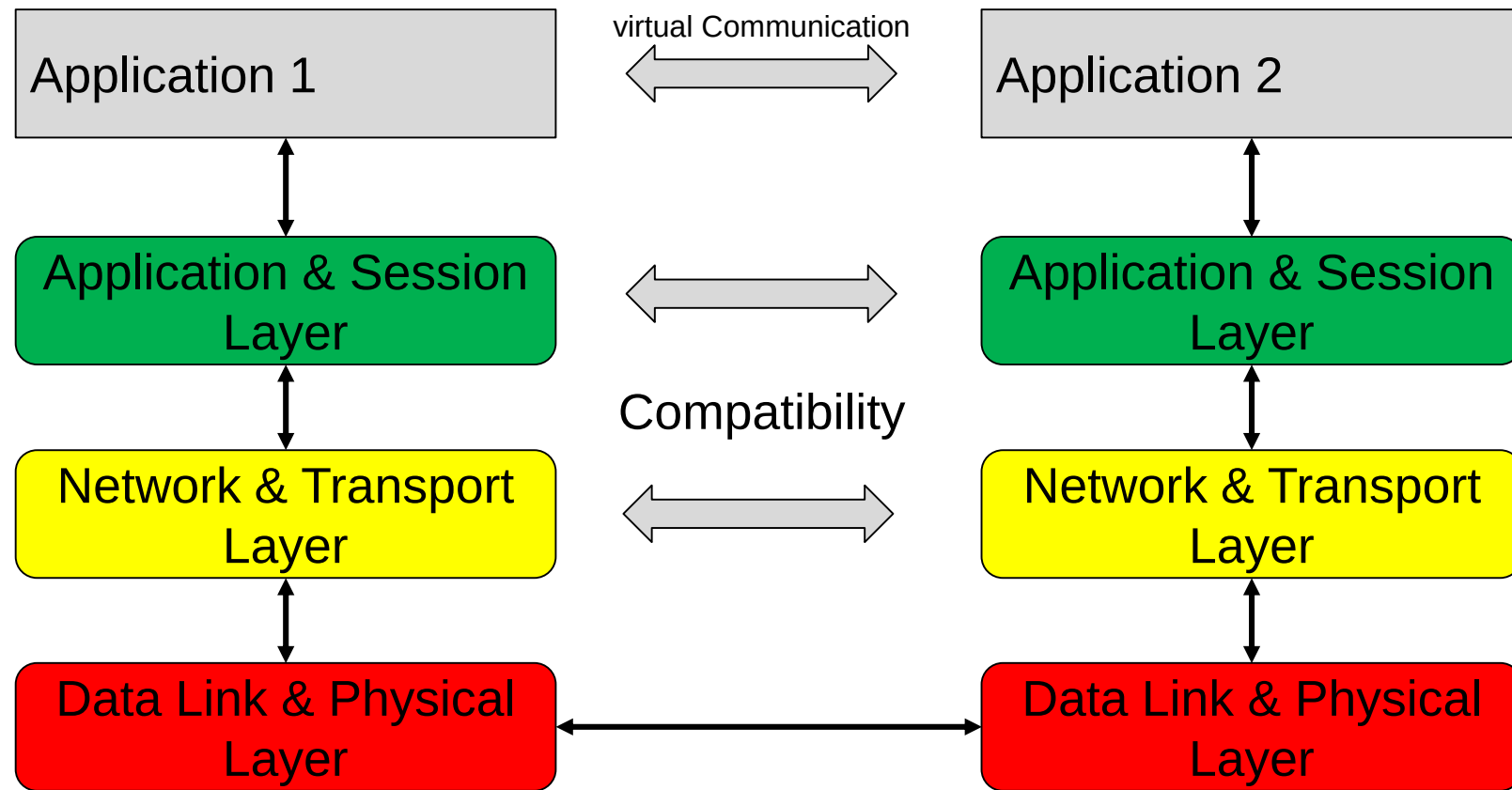


Well known model

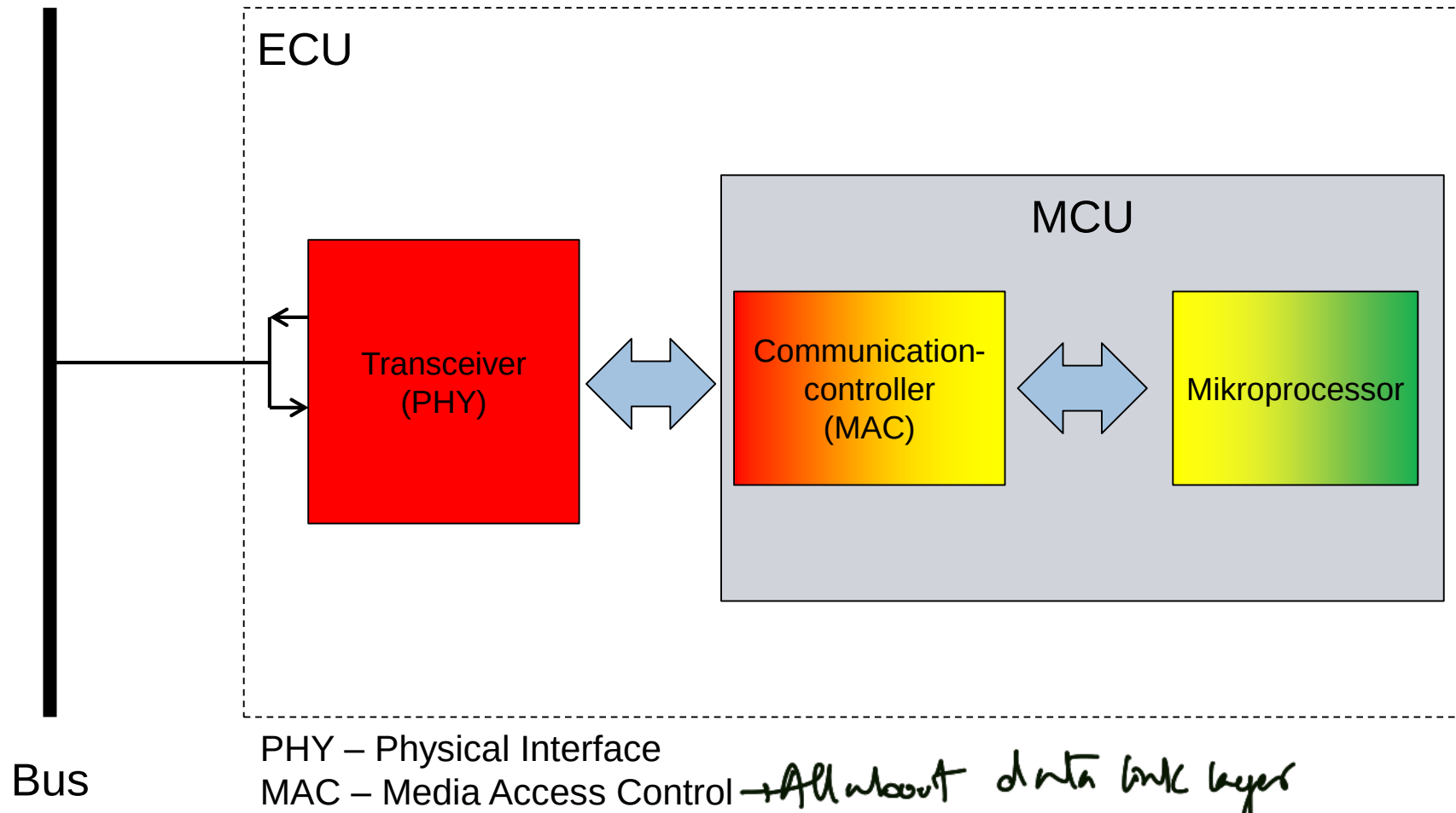
- **OSI/ISO – Open Systems Interconnection**
Reference Model
- Developed in 1977 – 1984 as theoretical (!)
reference model



Protocol Stack



Hardware Setup



Examples

Application & Session Layer

- KWP 2000, UDS, OBD (Diagnosis)
- CCP, XCP (Calibration)
- OSEK/VDX, AUTOSAR
- CANOpen

Network & Transport Layer

- ISO TP, AUTOSAR TP
- TP 1.6, 2.0 (VW)
- CANOpen

Data Link & Physical Layer

- I2C, SPI
- K-Line, LIN, RS232, RS422
- CAN, Flexray
- MOST
- Ethernet

Datarate classification (Automotive Industry)

- Bitrate $f_{Bit} = \frac{bits}{s}$ (Physical transfer rate)
- Datarate $f_{Data} = \frac{Number\ Userbits}{Number\ Userbits + Number\ Controlbits}$ (Protocol and channel overhead)

Class	Bitrate	Example	Application Area
Diagnosis	< 10Kbit/s	K-Line	Garage tester
A	< 25Kbit/s	LIN	Body and Chassis
B	25 ... 125kbit/s	CAN Low Speed	
C	125 ... 1000Kbit/s	CAN High Speed	Powertrain, Flash,...
D	> 1Mbit/s	Flexray	X by Wire
Infotainment	> 10Mbit/s	MOST, Ethernet	Multimedia

Baudrate: Number of symbols per second.

Attention: Symbol often is a bit, but may also be something different

Transfer types

Character based

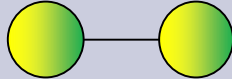
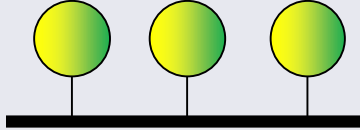
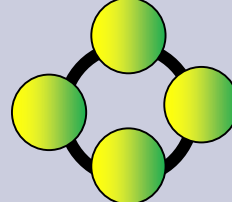
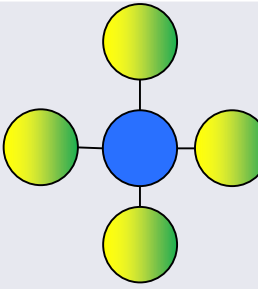
- Transmission of one character after the other
- Simple hardware
- Protocol implementation completely in software
- Large overhead (>20%)
- Example: UART

Bitstream based

- Transmission of larger packages
- More complex hardware (e.g. collision detection)
- Transport protocol in software
- Example: CAN, Ethernet

→ In bitstream based, the MAC is receiving a couple of characters and put these into a package/s.
Eg: In the case of CAN it is 8 bytes of Ethernet 1K bytes.

Topology

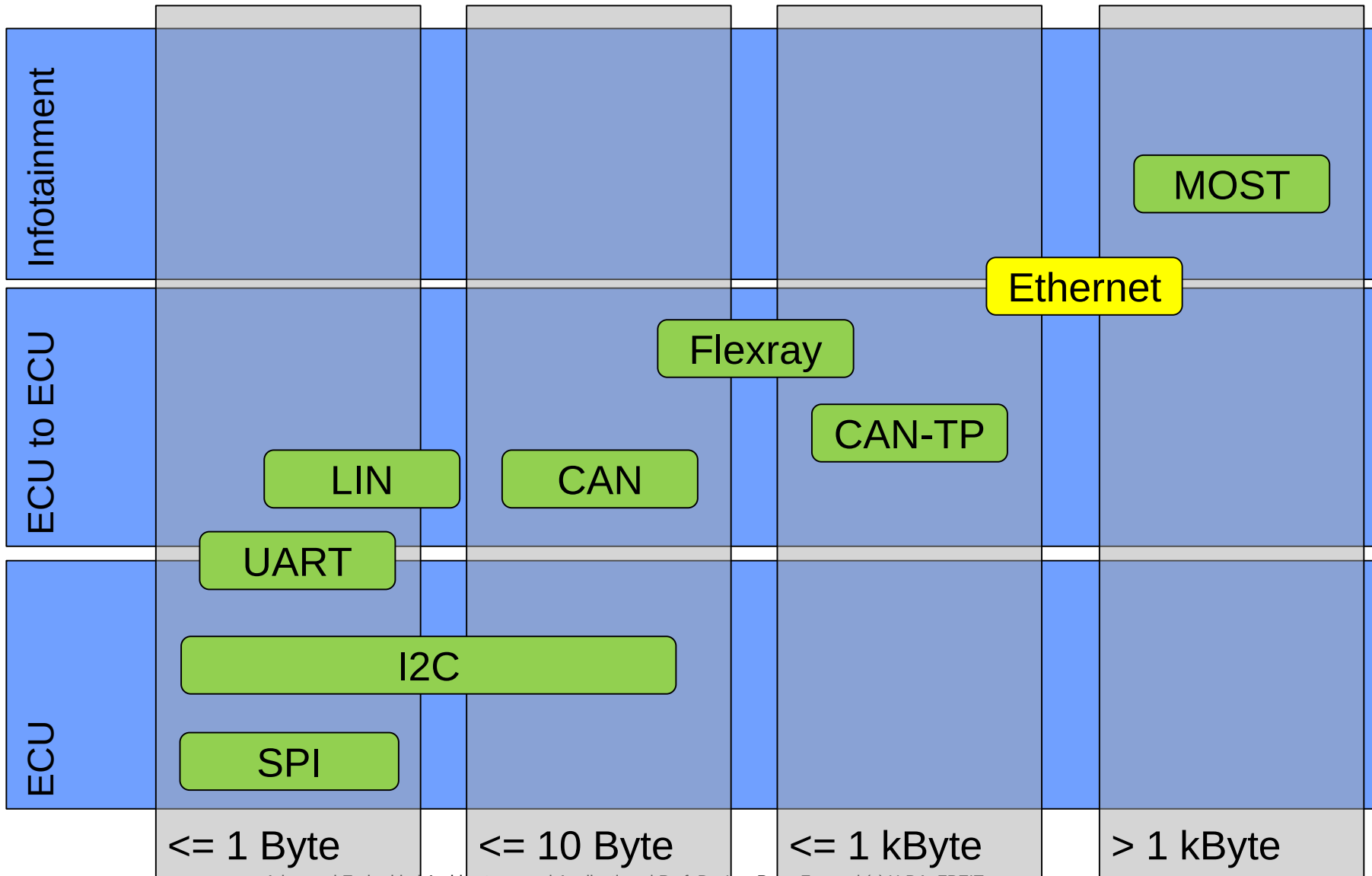
Type	Topology	Example
Point to Point		• RS232 • K-Line
Bus		• CAN • SPI • I2C • FlexRay
Ring		• Most
Star		• FlexRay

→ if it breaks, all of them will be offline

→ if bus breaks, only one of them is offline.

Physical topology and logical topology may differ!

Many networks available



Network comparison

Bus	Transmission Rate	Wire length	Standard	Costs	Robustness
SPI	KBit/s – MBit/s	Centimeter	de-facto	low	Low
I2C	Up to 400KBit/s v2.0 bis 3.4MBit/s	Centimeter	yes	low	low
USART	< 1MBit	Centimeter up to a few meters	de-facto	low	medium
LIN	1..20KBit/s	Few meters	yes	Low - medium	medium
CAN	Up to 1MBit/s	Several meter	yes inkl. TP	medium	high
Flexray	Up to 10MBit/s	Several meter	yes	Medium - high	Very high
MOST	Up to 150MBit/s	Few meter	yes, all 7 ISO/OSI Layers	high	medium
Ethernet	Up to Gigabit	Several meter	de-facto	mittel	medium

Car Requirements

Id	Requirement
F1	Transmit Joystick data from labboard to car
F2	Establish connection state between joystick and car
NFR1	Range up to 10m
NFR2	Data range 9800Baud or more (10Byte per 10ms)
NFR3	Reliable detection of transmission errors

Zigbee

Zigbee is an IEEE 802.15.4-based specification for a suite of high-level communication protocols used to create personal area networks with small, low-power digital radios, such as for home automation, medical device data collection, and other low-power low-bandwidth needs, designed for small scale projects which need wireless connection. Hence, Zigbee is a low-power, low data rate, and close proximity (i.e., personal area) wireless ad hoc network.

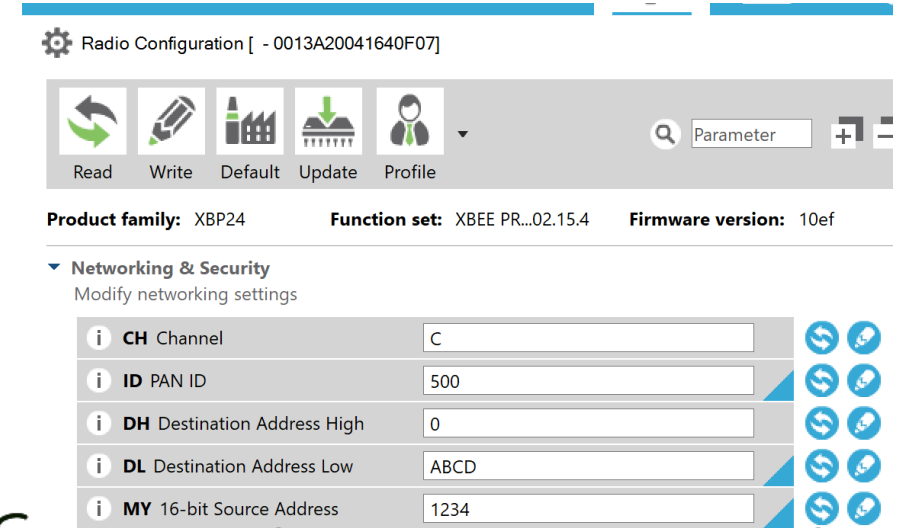
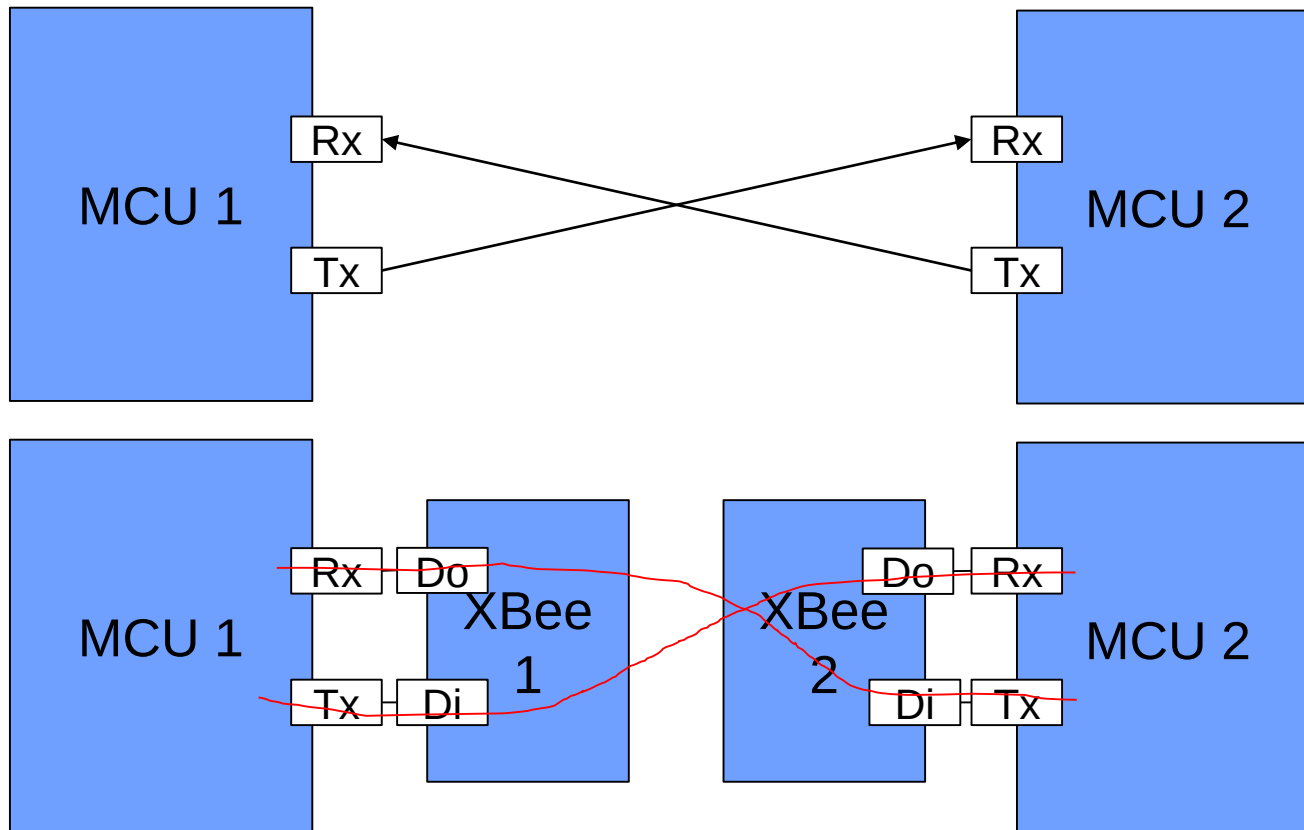
The technology defined by the Zigbee specification is intended to be simpler and less expensive than other wireless personal area networks (WPANs), such as Bluetooth or more general wireless networking such as Wi-Fi. Applications include wireless light switches, home energy monitors, traffic management systems, and other consumer and industrial equipment that requires short-range low-rate wireless data transfer.

Its low power consumption limits transmission distances to 10–100 meters line-of-sight, depending on power output and environmental characteristics. Zigbee has a defined rate of 250 kbit/s, best suited for intermittent data transmissions from a sensor or input device.

(Wikipedia)

Application

- The Xbee and Xbee pro modules act as a transparent wireless UART bridge
- Xbee modules communicate to other Xbee modules of the same network
- The network is defined by configuring the PAN ID and channel parameter



→ For using wifi module on psoc, it doesnot provide an wifi or ethernet mac. To implement that we need 2nd mc.

Our mission

Car Application

Remote Control Application

Protocol Stack

Protocol Stack

Zigbee 1

Zigbee 2

Our challenge

Xbee configuration

Parser

Protocol

ISR handling

Buffering



Big Bang will not work – NEVER!

With communication stacks (and other complex components) we usually only have a single bit of information after our first implementation.

The bastard is not working!

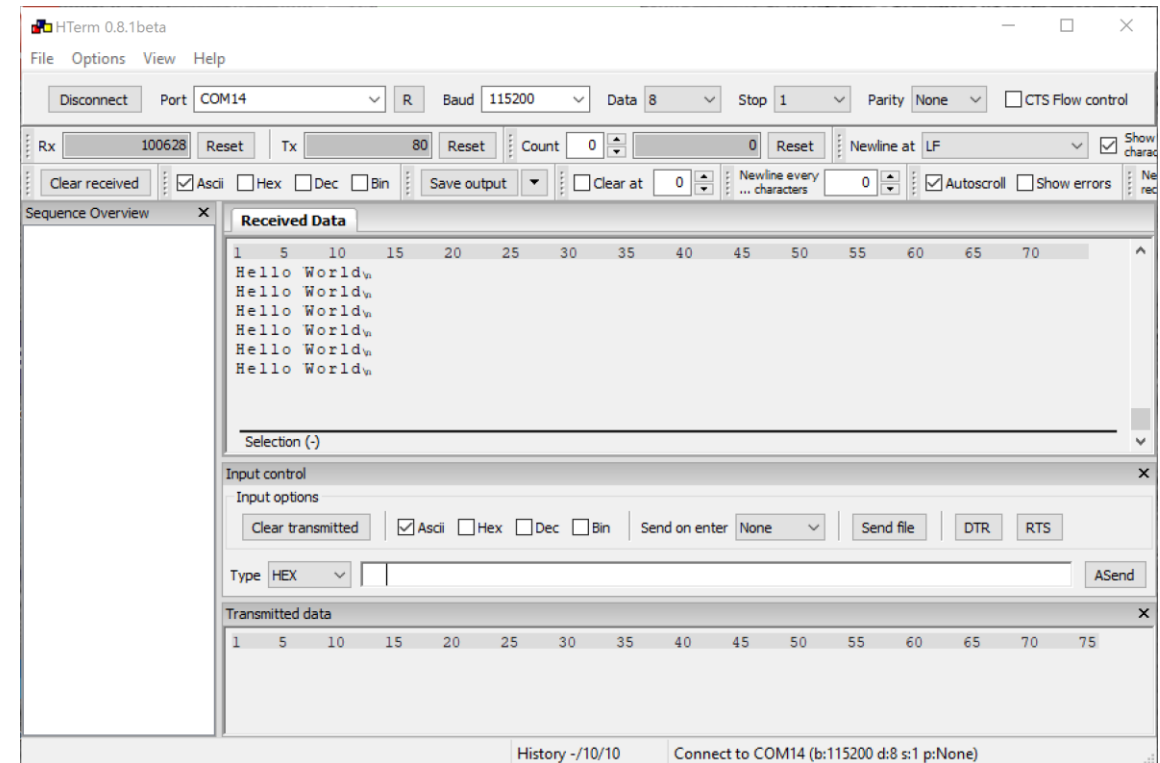
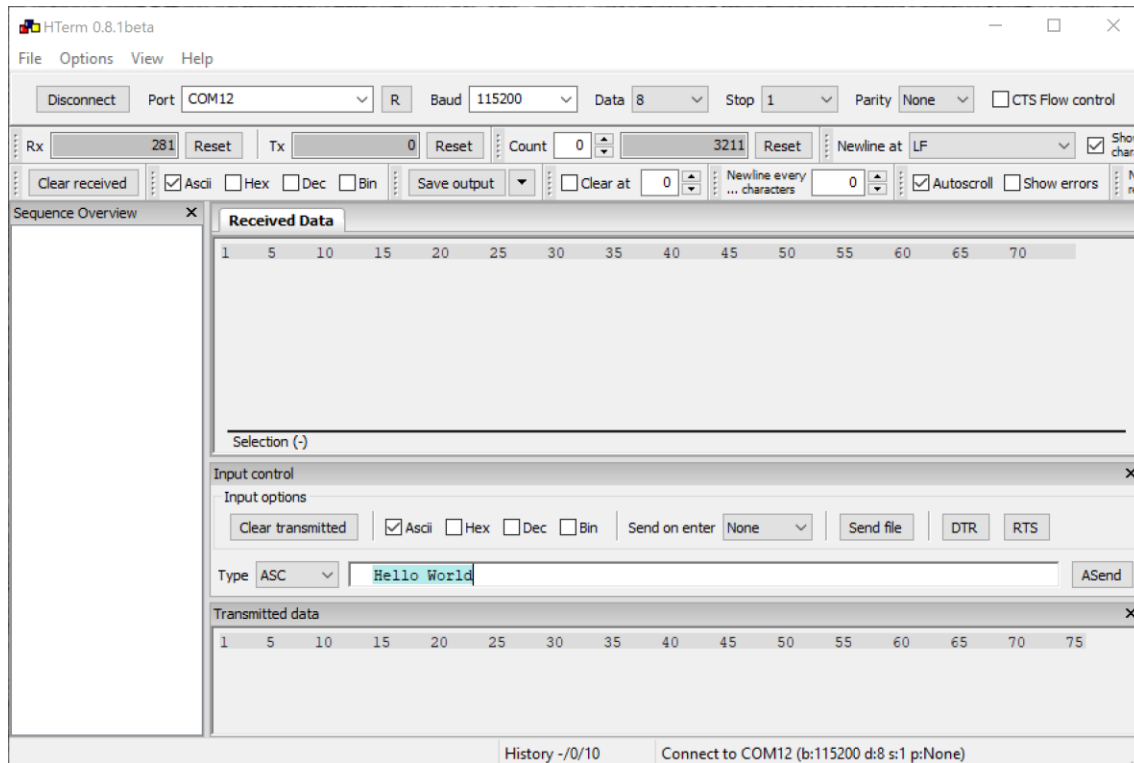
General engineering strategy

- Start with the low level component first
- Take out as many uncertainties as possible

Own code is a huge uncertainty!!!!

Strategy – Step 1 / Hello world

- Configure the 2 Xbee's for transparent communication mode
- Connect them both to your PC
- Open 2 Hterms and test the link



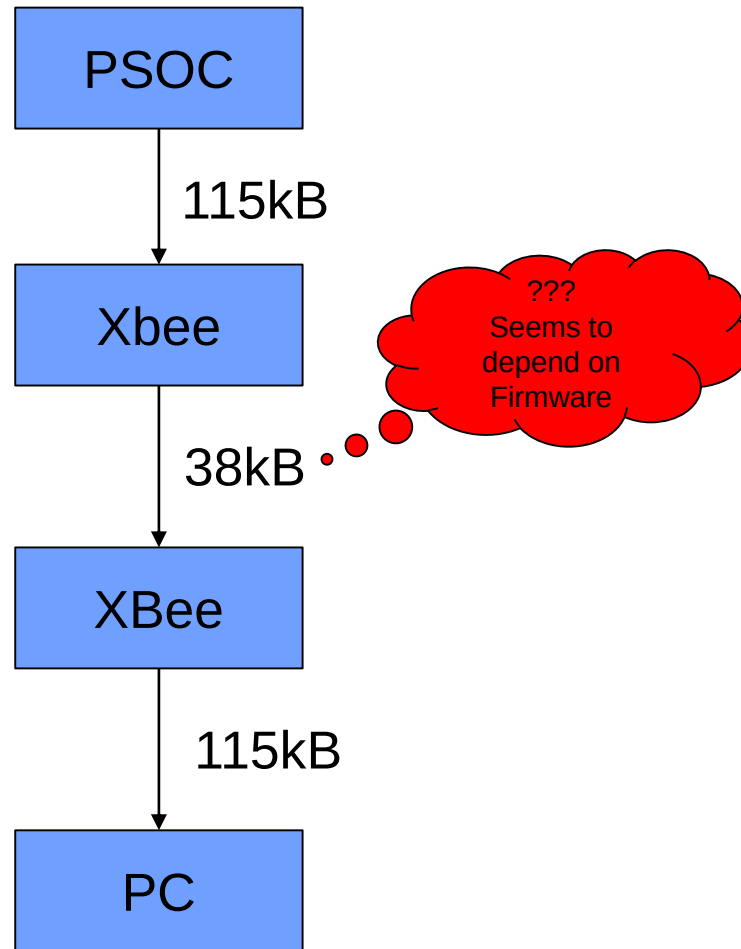
Strategy - Step 2 / Running on real hardware

- Replace one HTERM with the PSOC and simulate the other PSOC with HTERM
- Friendly test, followed by a first stress test ;-)

```
void REMOTE_testProtocol_run()
{
    for (int i = 0; i < TESTRUNS; i++)
    {
        UART_Zigbee_PutString("Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n");
    }
}
```

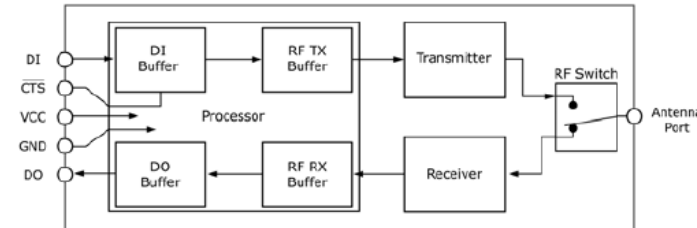
```
1   5   10  15  20  25  30  35  40  45  50  55  60  65  70
Hello World  2 3 4 5 6 7 8 9 100 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 199 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello Wrld 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 1 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 0 11 12 13 14 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 1 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello Wordd 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20\n
Hello Worldl 1 2 3 4 5 6 7 8 9 10 11 12 13 114 15 16 17 18 19 20\n
Hello World 1 2 3 4 5 6 7 8 9 10 1 12 13 14 15 16 17 18 19 20
```

Challenge – Buffering and Hardware Handshake



2.1.4. Flow Control

Figure 2-03. Internal Data Flow Diagram



DI (Data In) Buffer

When serial data enters the RF module through the DI pin (pin 3), the data is stored in the DI Buffer until it can be processed.

Hardware Flow Control (CTS). When the DI buffer is 17 bytes away from being full; by default, the module de-asserts $\overline{\text{CTS}}$ (high) to signal to the host device to stop sending data [refer to D7 (DIO7 Configuration) parameter]. $\overline{\text{CTS}}$ is re-asserted after the DI Buffer has 34 bytes of memory available.

How to eliminate the need for flow control:

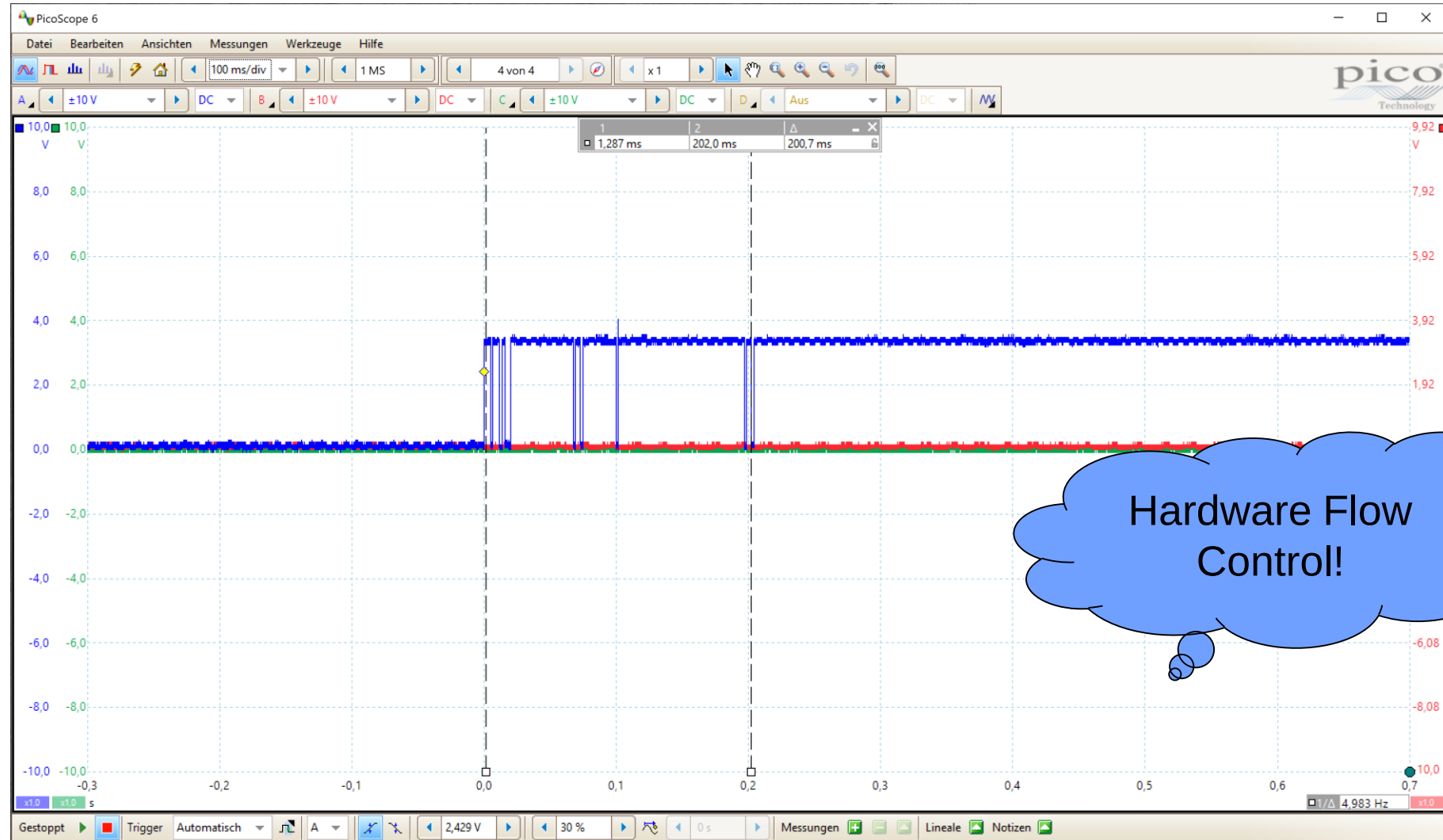
1. Send messages that are smaller than the DI buffer size.
2. Interface at a lower baud rate [BD (Interface Data Rate) parameter] than the throughput data rate.

Case in which the DI Buffer may become full and possibly overflow:

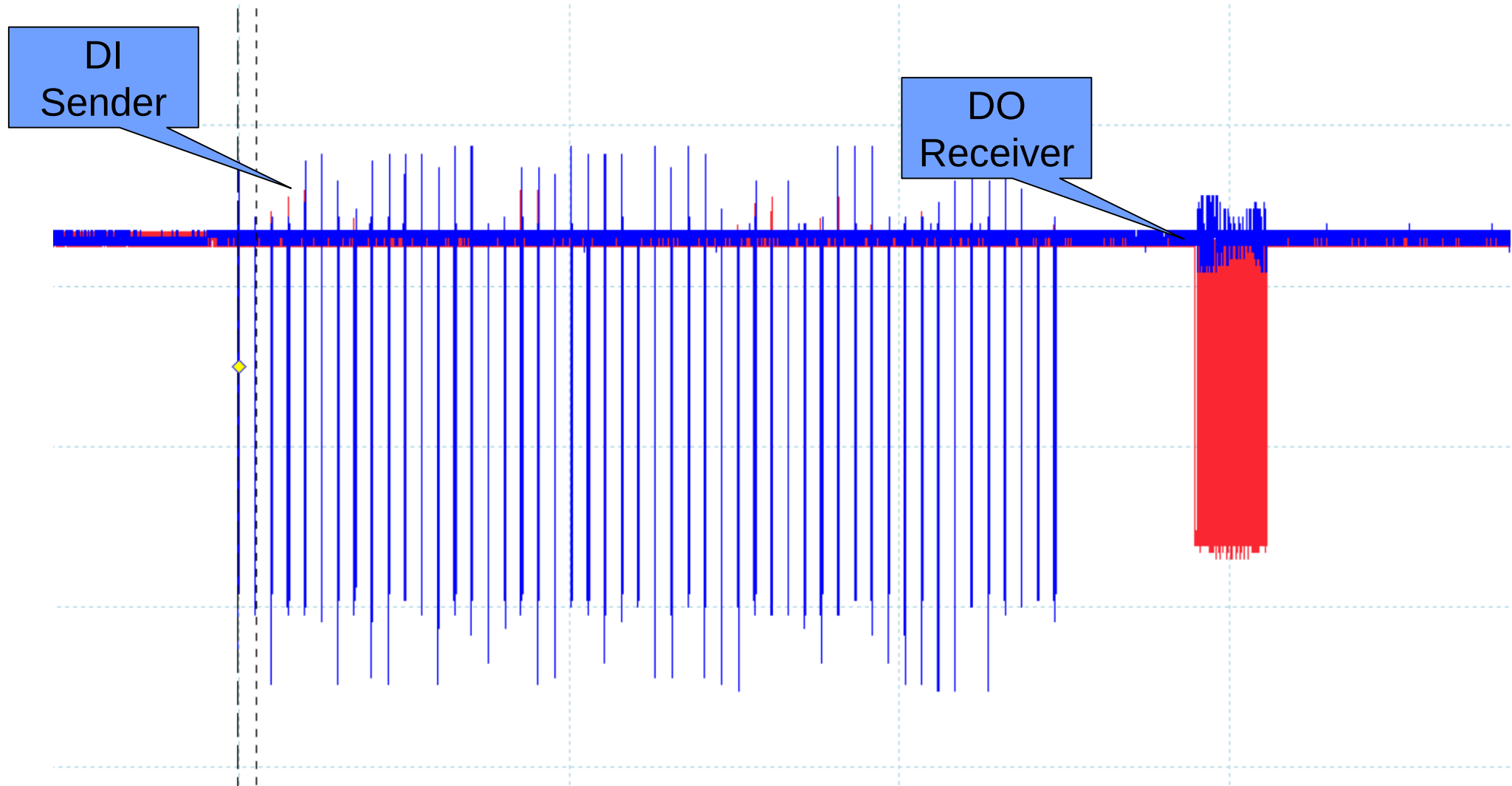
If the module is receiving a continuous stream of RF data, any serial data that arrives on the DI pin is placed in the DI Buffer. The data in the DI buffer will be transmitted over-the-air when the module is no longer receiving RF data in the network.

Refer to the RO (Packetization Timeout), BD (Interface Data Rate) and D7 (DIO7 Configuration) command descriptions for more information.

Stresstest – CTS goes high

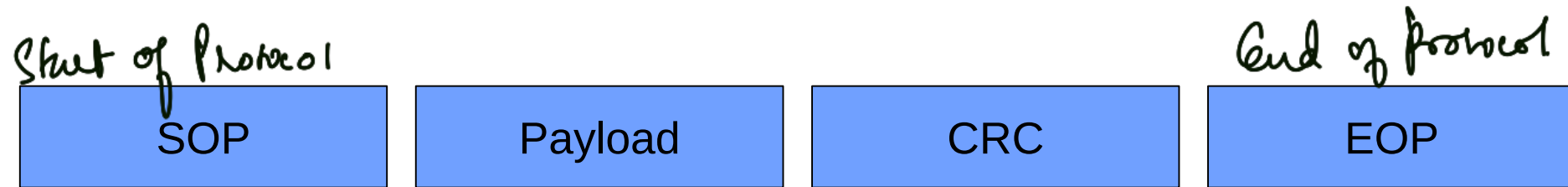


Zooming in – Testcase: Transmit 50 bytes, 1ms between 2 bytes



Strategy – Step 3 / Protocol Development

The protocol defines a set of data we want to transmit.

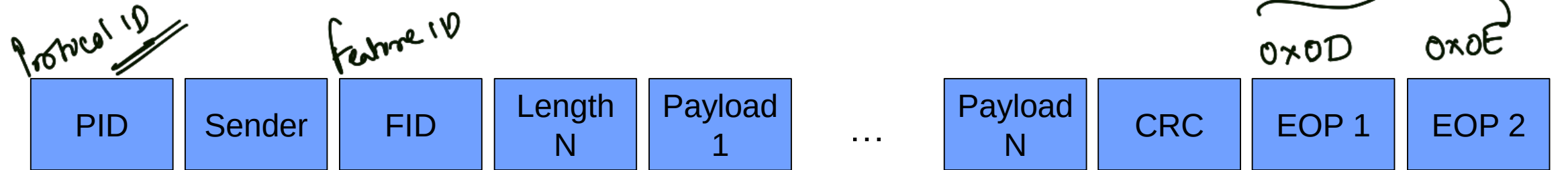


For our application, we have the following requirements:

- Continuous transmission of Joystick data (several bytes payload)
- Asynchronous transmission of carstate control data (one byte payload)

→ SOP & EOP - helps us to resync data if we are missing any.

Our Protocol Definition



- As we don't want to force the user to push all the bytes one after the other to the UART port, we will provide an abstract signal, representing a protocol.
- The signallayer scaler function will translate the protocol into a byte stream
- And perform additional services like checking the validity and calculating the CRC.

Signal Structure

typedef struct

```
{
    PROT_id_t m_id;
    PROT_sender_t m_sender;
    PROT_fid_t m_fid;
    uint8_t m_length;
    uint8_t m_payload[PROT_size_payload];
} SIGNAL_PROTOCOL_data_t;
```

typedef enum

```
{
    PROT_ID_Connect = 'C',
    PROT_ID_Joystick = 'J',
} PROT_id_t;
```

typedef enum

```
{
    PROT_sender_car = 'C',
    PROT_sender_remote = 'R',
} PROT_sender_t;
```

typedef enum

```
{
    PROT_FID_Connect = 'C',
    PROT_FID_Feature = 'F',
    PROT_FID_Disconnect = 'D',
    PROT_FID_Stop = 'S',
} PROT_fid_t;
```

Our Protocol Signal

```
typedef enum
{
    PROT_ID_Connection = 'C',    /**<\brief connection state related protocol */
    PROT_ID_Joystick    = 'J',    /**<\brief joystick data related protocol */
} PROT_id_t;

typedef enum
{
    PROT_FID_Connect      = 'C',    /**<\brief request connection (remote), confirm connection (car) */
    PROT_FID_Feature      = 'F',    /**<\brief request list of features (remote), provide features (car) */
    PROT_FID_Disconnect   = 'D',    /**<\brief request disconnect (remote), confirm disconnect (car) */
    PROT_FID_Stop         = 'S',    /**<\brief request emergency stop (remote) */
} PROT_fid_t;

typedef enum
{
    PROT_sender_car      = 'C',    /**<\brief car id */
    PROT_sender_remote   = 'R',    /**<\brief remote id */
} PROT_sender_t;

#define PROT_SIZEPAYLOAD 40        /**< \brief Payload size (max) */
#define PROT_SIZERXBUF   100       /**< \brief Size of the UART reception buffer */
#define PROT_EOP1        0x0D      /**< \brief EOP character 1 */
#define PROT_EOP2        0x0E      /**< \brief EOP character 2 */

typedef struct
{
    PROT_id_t      m_id;            /**< \brief Protocol ID, first byte in protocol */
    PROT_sender_t  m_sender;        /**< \brief identifier of the sender - mainly for debugging */
    PROT_fid_t     m_fid;          /**< \brief feature identifier */
    uint8_t        m_length;       /**< \brief length of the valid payload */
    uint8_t        m_payload[PROT_SIZEPAYLOAD]; /**< \brief payload */
} SIGNAL_PROTOCOL_data_t;
```

Implementation as Asynchronous Driver

Remember the MCAL story:

- We don't know when we will receive data
- We don't know when we can send data (minor problem)
- Furthermore our application will not process the data fast enough to avoid a loss of data under all circumstances!
- We need another FIFO-buffer.

```
ISR2(isr_Xbee_rx)
{
    uint8_t data = UART_Zigbee_GetByte();
    static uint8_t prevChar = 0;

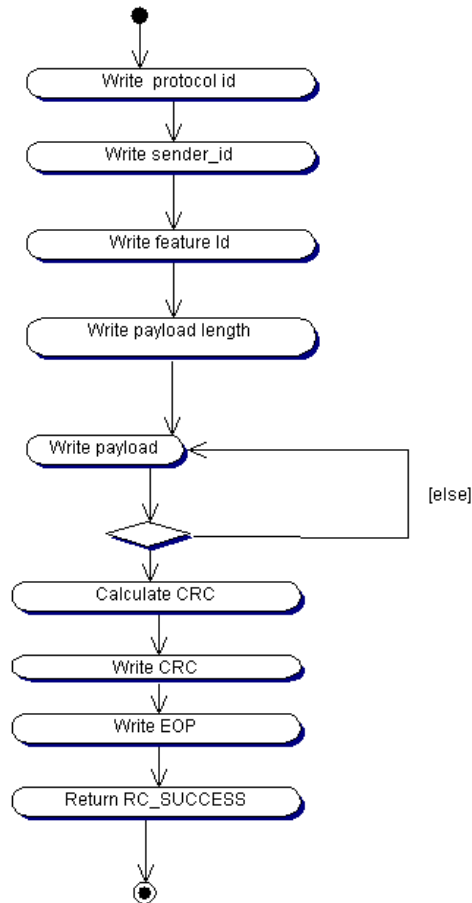
    //Todo: check for transmission errors
    //LOG_I("rx", "%x", data);

    RC_t result = Ringbuffer_Put(&REMOTE_rx_buffer, data);
    if (PROT_EOP1 == prevChar && PROT_EOP2 == data)
    {
        //Complete protocol
        SetEvent(tsk_EventDispatcher, ev_isr_protocolBufferComplete);
        prevChar = 0;
    }
    else
    {
        prevChar = data;
    }
}
```

Translating the protocol into a byte stream

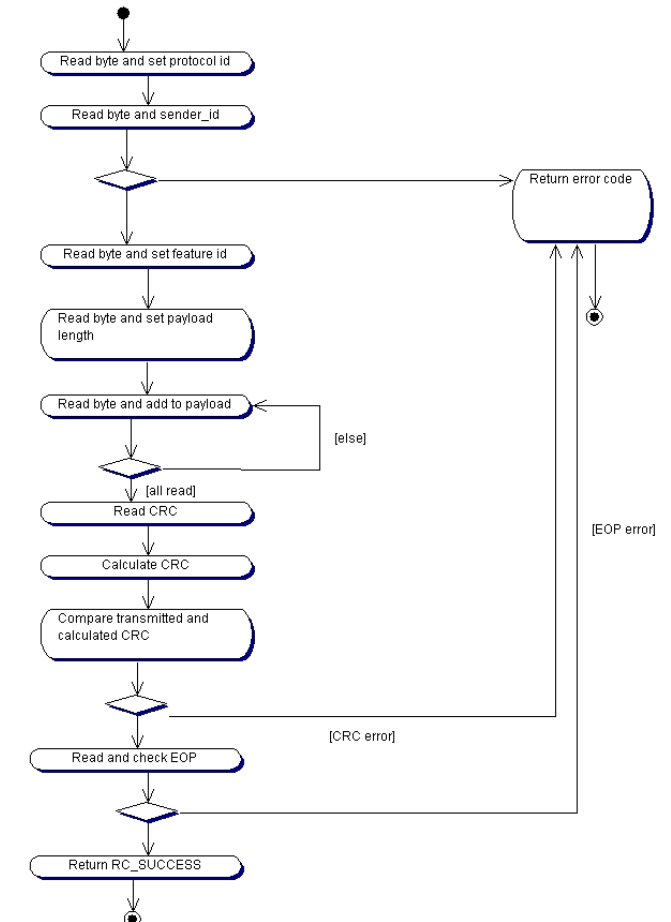
Transmitting

Bytes send to the buffer (UART internal). Errors writing to ringbuffer are not shown.



Receiving

Bytes are read from ringbuffer, checked for validity and stored in the signal. Errors reading ringbuffer are not shown.



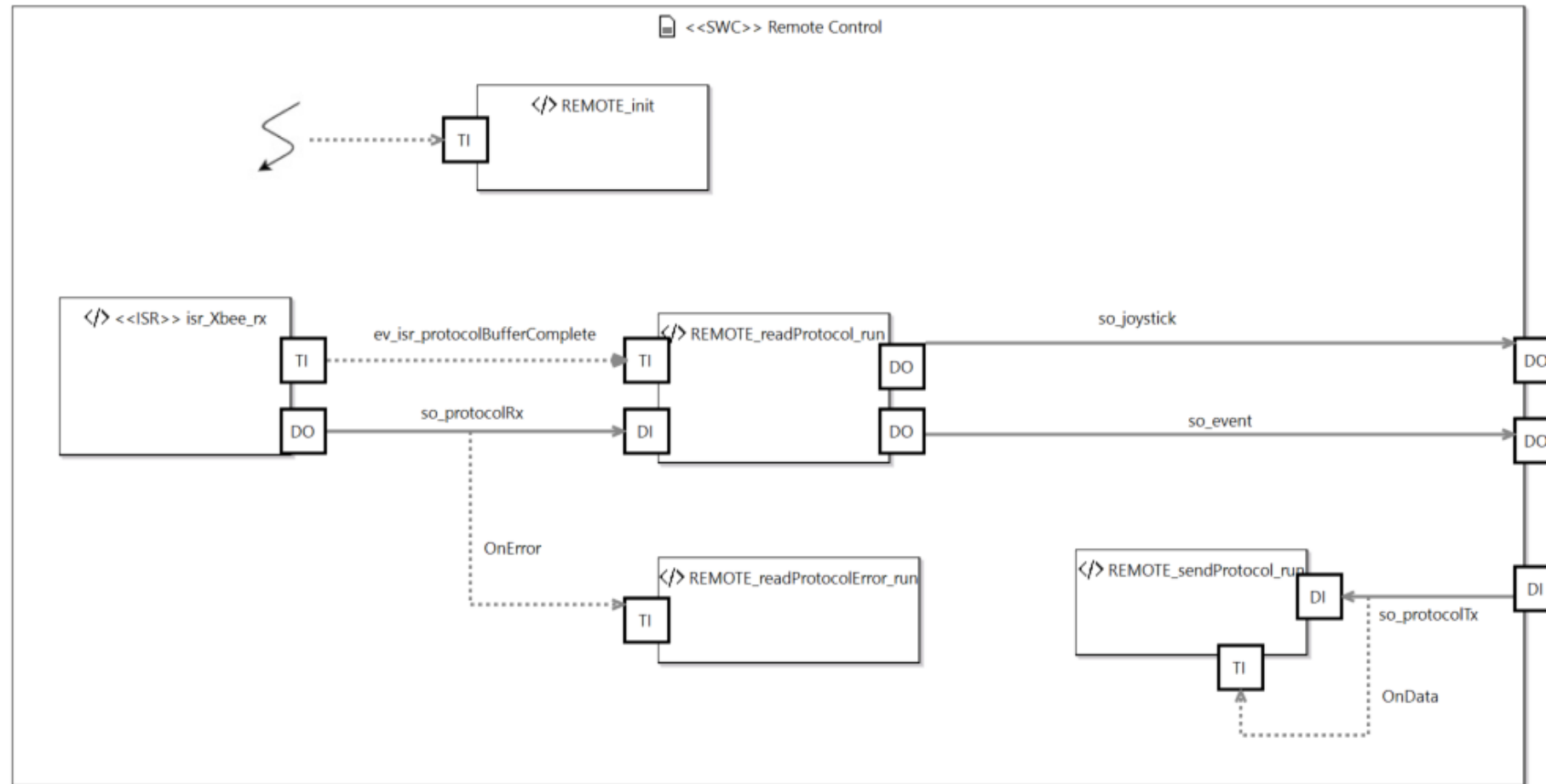
Cyclic Redundancy Check (CRC) calculation

A cyclic redundancy check (CRC) is an error-detecting code commonly used in digital networks and storage devices to detect accidental changes to raw data. Blocks of data entering these systems get a short check value attached, based on the remainder of a polynomial division of their contents. On retrieval, the calculation is repeated and, in the event the check values do not match, corrective action can be taken against data corruption. CRCs can be used for error correction (see bitfilters).

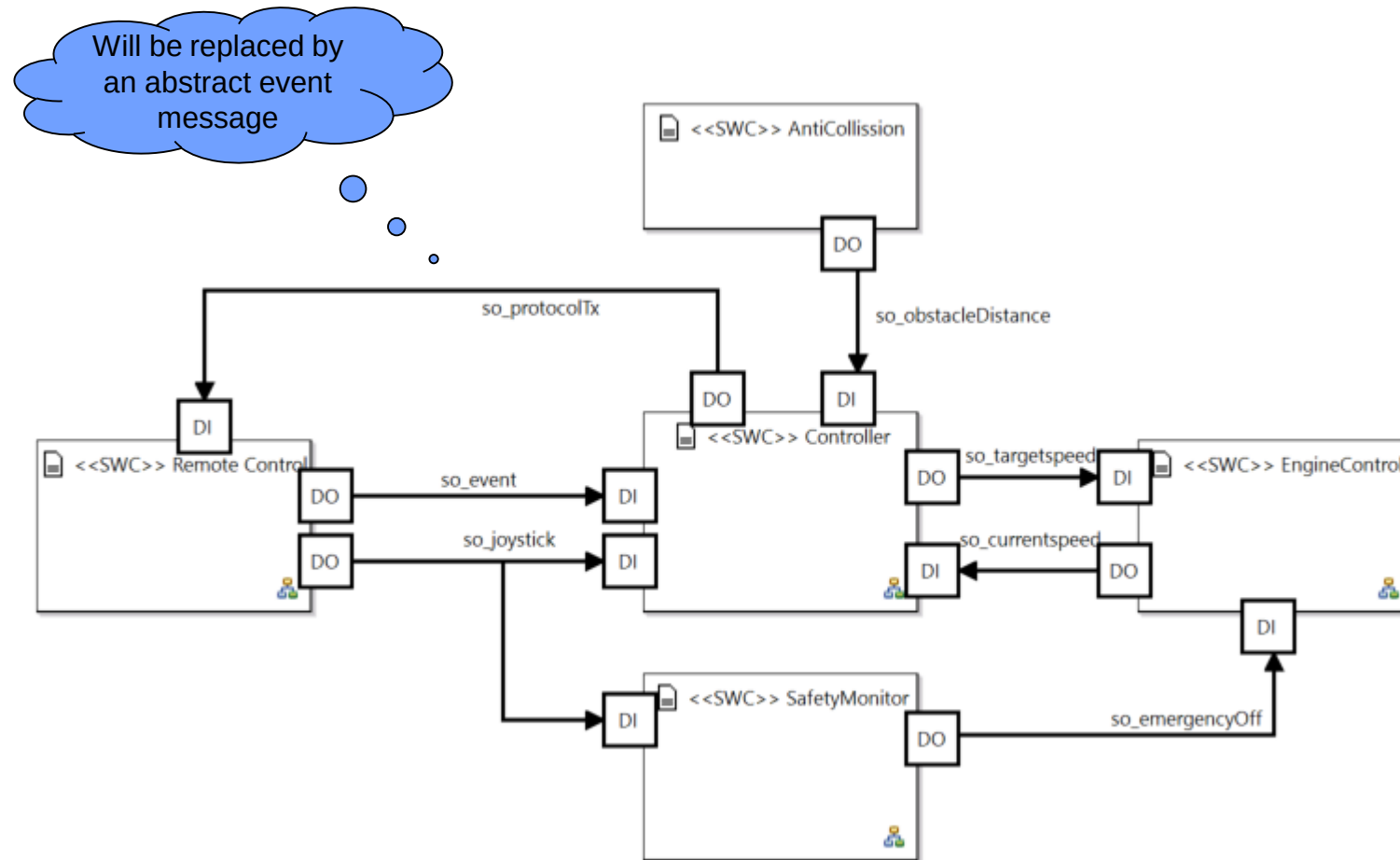
CRCs are so called because the check (data verification) value is a redundancy (it expands the message without adding information) and the algorithm is based on cyclic codes. CRCs are popular because they are simple to implement in binary hardware, easy to analyze mathematically, and particularly good at detecting common errors caused by noise in transmission channels. Because the check value has a fixed length, the function that generates it is occasionally used as a hash function.

The CRC was invented by W. Wesley Peterson in 1961; the 32-bit CRC function, used in Ethernet and many other standards, is the work of several researchers and was published in 1975. (Wikipedia)

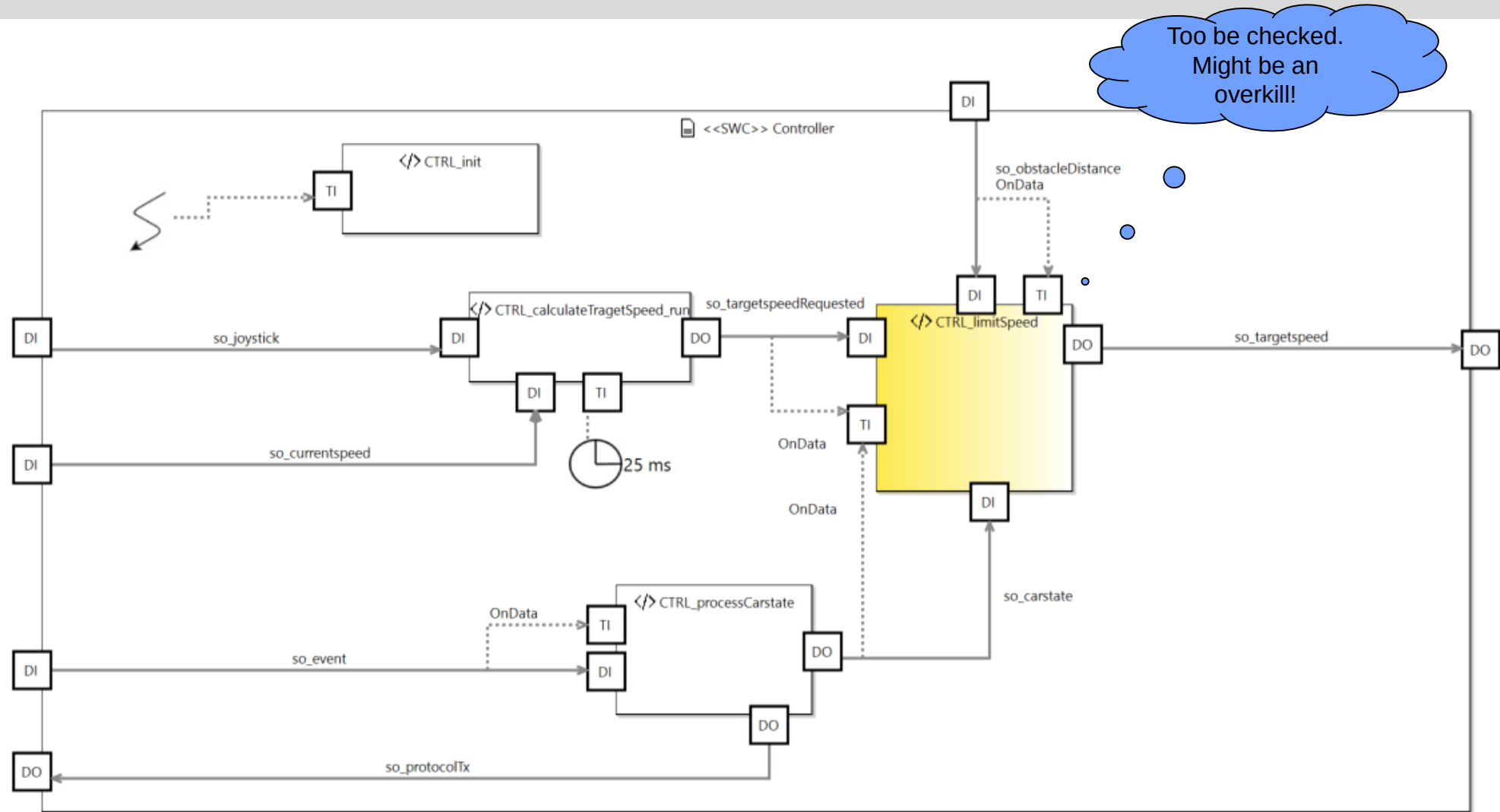
Integration into our system



Concept for the control architecture (First draft)



Concept for the control architecture



Let's do it...



Wrapup

- General introduction into communication networks and principles
- A closer look at the Xbee
- Protocol and Parser development
- Integration into the RTE architecture
- Now the real application development can start!